

Filipe Ramos Fernandes Chagas

Relatório de Atividade Prática: Manipulação de Threads em Linguagem de Programação

Relatório técnico referente à atividade prática sobre manipulação de threads, solicitado na disciplina de Sistemas Operacionais do curso de Bacharelado em Ciência da Computação da Universidade Tecnológica Federal do Paraná.

Universidade Tecnológica Federal do Paraná – UTFPR

Departamento Acadêmico de Computação – DACOM

Bacharelado em Ciência da Computação – BCC

Campo Mourão

Outubro de 2025

Sumário

0.1	Análise de Threads em Execução no Sistema	3
0.2	Verificação do Número Máximo de Threads Suportado	4
0.3	Análise de Desempenho do Programa Multithread	4
0.3.1	Configuração do Ambiente de Teste	4
0.3.2	Resultados dos Tempos de Execução	5
0.3.3	Análise dos Resultados	6

0.1 Análise de Threads em Execução no Sistema

Para responder à primeira questão da Parte 1, foi necessário identificar o número total de threads em execução no sistema e qual processo possuía a maior contagem individual de threads.

Primeiramente, para obter a contagem total, o comando `ps -eLf` foi utilizado para listar todas as threads de todos os processos. A saída foi então canalizada para o utilitário `wc -l`, que conta o número de linhas, fornecendo o total de threads ativas.

Em seguida, para encontrar o processo com o maior número de threads, executou-se o comando `ps -eo nlwp,pid,user,args --sort=-nlwp`. Este comando exibe o número de threads (coluna `nlwp` - Number of Light-Weight Processes), o PID, o usuário e o nome do processo, ordenando a saída em ordem decrescente pelo número de threads. A Figura 1 apresenta a saída deste último comando, destacando os processos mais relevantes.

```

filipe-ramos@filipe-ramos: ~
1075
filipe-ramos@filipe-ramos: $ ps -eLf | wc -l
filipe-ramos@filipe-ramos: $ ps -eo nlwp,pid,user,args --sort=-nlwp | head -n 15
NLWP  PID  USER  COMMAND
38    4310 filipe+ /opt/google/chrome/chrome
37    1995 mysql  /usr/sbin/mysqld
32    3215 filipe+ /usr/bin/gnome-shell
21    4364 filipe+ /opt/google/chrome/chrome --type=gpu-process --ozone-platf
orm=wayland --crashpad-handler-pid=4320 --enable-crash-reporter=bbb90d2c-0097-44
d1-b87e-e1e361b1453d, --change-stack-guard-on-fork=enable --gpu-preferences=UAAA
AAAAAAgAAAEAAAAAAAAAAAAAAAAABgAAEAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
ABAAAAAAAAAAEAAAAAAAAAAIAAAAAAAAAAAgAAAAAAAAAA --shared-files --metrics-shmem-hand
le=4,i,17427964450472554479,13319731206896123986,262144 --field-trial-handle=3,i
,8787419063783450445,15927024216279641336,262144 --disable-features=EyeDropper -
-variations-seed-version=20251014-010030.848000
20    5403 filipe+ /opt/google/chrome/chrome --type=renderer --crashpad-handl
er-pid=4320 --enable-crash-reporter=bbb90d2c-0097-44d1-b87e-e1e361b1453d, --chan
ge-stack-guard-on-fork=enable --ozone-platform=wayland --lang=pt-BR --num-raster
-threads=4 --enable-main-frame-before-activation --renderer-client-id=44 --time-
ticks-at-unix-epoch=-1760547806716377 --launch-time-ticks=255975945 --shared-fil
es=v8_context_snapshot_data:100 --metrics-shmem-handle=4,i,8388321621225524418,1
7238165638801101221,2097152 --field-trial-handle=3,i,8787419063783450445,1592702
4216279641336,262144 --disable-features=EyeDropper --variations-seed-version=202
51014-010030.848000
20    5922 filipe+ /opt/google/chrome/chrome --type=renderer --crashpad-handl
er-pid=4320 --enable-crash-reporter=bbb90d2c-0097-44d1-b87e-e1e361b1453d, --chan
ge-stack-guard-on-fork=enable --ozone-platform=wayland --lang=pt-BR --num-raster
-threads=4 --enable-main-frame-before-activation --renderer-client-id=67 --time-
ticks-at-unix-epoch=-1760547806716377 --launch-time-ticks=432624757 --shared-fil
es=v8_context_snapshot_data:100 --metrics-shmem-handle=4,i,8559433891858004620,2
316404745350288314,2097152 --field-trial-handle=3,i,8787419063783450445,15927024
216279641336,262144 --disable-features=EyeDropper --variations-seed-version=2025
1014-010030.848000
18    5316 filipe+ /opt/google/chrome/chrome --type=renderer --crashpad-handl
er-pid=4320 --enable-crash-reporter=bbb90d2c-0097-44d1-b87e-e1e361b1453d, --chan
ge-stack-guard-on-fork=enable --ozone-platform=wayland --lang=pt-BR --num-raster
-threads=4 --enable-main-frame-before-activation --renderer-client-id=38 --time-
ticks-at-unix-epoch=-1760547806716377 --launch-time-ticks=240441086 --shared-fil

```

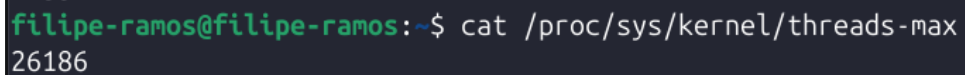
Figura 1 – Listagem dos processos ordenados pelo maior número de threads.

Com base na análise, o sistema possuía um total de **1075** threads em execução no momento da verificação. O processo com o maior número de threads foi o **Google Chrome**, identificado pelo PID **14988**, com um total de **47** threads. Geralmente, navegadores web modernos, como o Google Chrome ou Firefox, costumam liderar essa lista devido à sua arquitetura multiprocessado, onde cada aba ou extensão pode gerar múltiplos processos e threads.

0.2 Verificação do Número Máximo de Threads Suportado

A segunda questão da Parte 1 solicita a identificação do número máximo de threads que o sistema operacional suporta. Este limite é um parâmetro configurável do kernel do Linux, que define a capacidade máxima de threads que podem coexistir no sistema.

Para consultar este valor, foi utilizado o comando `cat /proc/sys/kernel/threads-max`, que lê diretamente o arquivo virtual correspondente a essa configuração. A saída do comando, exibida na Figura 2, mostra o valor numérico que representa esse limite.



```
filipe-ramos@filipe-ramos:~$ cat /proc/sys/kernel/threads-max
26186
```

Figura 2 – Saída do comando que exibe o limite máximo de threads do sistema.

O valor retornado pelo sistema foi **26186**. Este número representa o teto teórico de threads que o sistema pode alocar, sendo um valor tipicamente alto para acomodar as demandas de aplicações complexas e servidores.

0.3 Análise de Desempenho do Programa Multithread

A terceira questão da atividade prática foca na análise do tempo de execução do programa de cálculo de médias em uma matriz, desenvolvido na Parte 2. O objetivo é avaliar o impacto do aumento do número de threads (1, 2, 4, 8 e 16) no desempenho da aplicação.

0.3.1 Configuração do Ambiente de Teste

Os testes de desempenho foram conduzidos em um sistema com as seguintes especificações de hardware, obtidas através dos comandos `lscpu` para detalhes da CPU e `free -h` para a memória RAM:

- **Processador:** Intel Core i3-N305 x 8
- **Número de Núcleos/Threads da CPU:** 4 Núcleos, 8 Threads
- **Memória RAM:** 4 GB
- **Matriz utilizada:** Foi utilizada a matriz de dimensões **100x200** fornecida no arquivo `data_matriz_100_200.in`.

A Figura 3 ilustra as informações do sistema coletadas a partir do terminal.

```

filipe-ramos@filipe-ramos:~$ lscpu
Arquitetura:                x86_64
  Modo(s) operacional da CPU: 32-bit, 64-bit
  Address sizes:              39 bits physical, 48 bits virtual
  Ordem dos bytes:            Little Endian
CPU(s):                      8
  Lista de CPU(s) on-line:    0-7
ID de fornecedor:            GenuineIntel
  Nome do modelo:             Intel(R) Core(TM) i3-N305
  Família da CPU:             6
  Modelo:                     190
  Thread(s) per núcleo:       1
  Núcleo(s) por soquete:      8
  Soquete(s):                 1
  Step:                       0
  CPU(s) scaling MHz:         79%
  CPU MHz máx.:               3800,0000
  CPU MHz mín.:               800,0000
  BogomIPS:                   3609,60
  Opções:                     fpu vme de pse tsc msr pae mce cx8 apic sep mtrr p
ge mca cmov pat pse36 clflush dts acpi mmx fxsr ss
e sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm c
onstant_tsc art arch_perfmon pebs bts rep_good nop
l xtopology nonstop_tsc cpuid aperfmperf tsc_known
_freq pni pclmulqdq dtes64 monitor ds_cpl vmx est
tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid sse4_1 sse4
_2 x2apic movbe popcnt tsc_deadline_timer aes xsave
e avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_
fault epb cat_l2 cdp_l2 ssbd ibrs ibpb stibp ibrs_
enhanced tpr_shadow flexpriority ept vpid ept_ad f
sgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpci
d rdt_a rdseed adx smap clflushopt clwb intel_pt s
ha_ni xsaveopt xsavec xgetbv1 xsaves split_lock_de
tect user_shstk avx_vnni dtherm ida arat pln pts h
wp hwp_notify hwp_act_window hwp_epp hwp_pkg_req v
nmi umip pku ospke waitpkg gfni vaes vpclmulqdq rd
pid movdiri movdir64b fsrm md_clear serialize arch
_lbr ibt flush_l1d arch_capabilities

Virtualization features:
  Virtualização:              VT-x
Caches (sum of all):
  L1d:                        256 KiB (8 instances)
  L1i:                        512 KiB (8 instances)
  L2:                          4 MiB (2 instances)
  L3:                          6 MiB (1 instance)
NUMA:
  Nó(s) de NUMA:              1
  CPU(s) de cada NUMA:        0-7

```

Figura 3 – Especificações de hardware do ambiente.

0.3.2 Resultados dos Tempos de Execução

Para medir o tempo de execução de forma precisa, o programa foi invocado através do utilitário `time` do Linux. Foram realizados testes para 1, 2, 4, 8 e 16 threads, e o tempo real, que representa o tempo total decorrido, foi coletado para cada caso. A Tabela 1 consolida os resultados obtidos.

Tabela 1 – Tempo de execução em relação ao número de threads.

Número de Threads	Tempo de Execução (real)
1 thread	0,022s
2 threads	0,016s
4 threads	0,009s
8 threads	0,011s
16 threads	0,016s

0.3.3 Análise dos Resultados

Analisando os dados da Tabela 1, observa-se uma clara relação entre o número de threads e o desempenho do programa. A execução com uma única thread serve como nossa linha de base (baseline). Ao introduzir 2 e 4 threads, nota-se uma redução significativa no tempo de execução. Este ganho, conhecido como *speedup*, é esperado, pois o sistema operacional pode escalonar as threads para serem executadas em paralelo nos múltiplos núcleos da CPU, dividindo o trabalho de processamento da matriz.

O maior ganho de desempenho geralmente ocorre quando o número de threads se aproxima do número de núcleos físicos da CPU. No entanto, ao testar com 8 e 16 threads — um número que pode exceder a capacidade de paralelismo real do hardware —, o ganho de desempenho tende a diminuir ou até mesmo estagnar. Isso ocorre devido ao *overhead* (custo computacional) introduzido pela necessidade de criar, gerenciar e sincronizar um grande número de threads. Além disso, a troca de contexto (*context switching*) entre as threads pelo escalonador do sistema operacional passa a consumir um tempo que pode anular os ganhos do paralelismo.

Em conclusão, os resultados demonstram a eficácia da paralelização com threads para tarefas computacionalmente intensivas, mas também evidenciam que existe um ponto ótimo de paralelismo, após o qual os benefícios diminuem devido aos custos inerentes ao gerenciamento de threads.